RELATÓRIO TÉCNICO DE PROJETO PRÁTICO

Tema C — Sistema de Gestão de Biblioteca

Disciplina: Estrutura de Dados | Curso: Ciência da Computação / Engenharia de Software

Docente: Prof. Me. Heber Castro | **Discente:** Yuri

Ano Letivo / Semestre: 2026 / 2

Sumário

1. Introdução e Objetivo do Sistema
2. Modelagem dos Dados, Structs e Campos
3. Justificativa Técnica da Escolha das Estruturas de Dados
4. Árvore Binária de Busca (BST) e Análise de Casos de Remoção
5. Grafo, Mapeamento Compacto, Rollback e Busca em Largura (BFS)
6. Estruturas Auxiliares, Histórico e Algoritmo Próprio de Ordenação
7. Tabelas de Análise de Complexidade Temporal e Espacial (Big-O)
8. Relatório Completo dos Testes Reproduzíveis
9. Conclusão e Limitações Conhecidas
10. Referências Bibliográficas
11. Declaração Transparente do Uso de Ferramentas de Inteligência Artificial

1. Introdução e Objetivo do Sistema

O gerenciamento de acervos bibliográficos modernos exige o processamento coordenado de múltiplos relacionamentos entre registros textuais e numéricos. No domínio de uma biblioteca (Tema C), uma aplicação de catálogo eficiente precisa atender simultaneamente a três requisitos fundamentais:

1. **Recuperação rápida e determinística** de obras por chave primária de identificação única (código de tombamento / ID);
2. **Mapeamento explícito de relações temáticas** ou de co-leitura entre livros (recomendações de obras correlatas, trilhas de estudo e análise de proximidade em rede);
3. **Auditabilidade estrita** de alterações de estado do acervo com garantia de consistência relacional e controle de concorrência/capacidade.

O objetivo deste projeto prático individual é projetar, implementar e validar formalmente um sistema completo de gestão de biblioteca em linguagem C (padrão C99 estrito), utilizando estruturas de dados fundamentais sem o emprego de facilitadores de alto nível ou bibliotecas externas.

O sistema integra uma **Árvore Binária de Busca (BST)** para indexação primária por ID, um **Grafo Não Direcionado** com matriz de adjacência e tabela de dispersão/mapeamento de slots para modelar afinidades entre livros por ID arbitrário, **Busca em Largura (BFS)** apoiada por **Fila Dinâmica FIFO** e **Lista Simplesmente Encadeada** (com inserção no fim em O(1) e vizinhos em ordem crescente de ID), uma **Pilha Dinâmica Encadeada** para histórico LIFO de alterações concluídas com sucesso, algoritmos próprios de **Merge Sort** e **Quick Sort** sobre cópias em vetor auxiliar, e camada de **Entrada Segura de Dados (I/O)** com prevenção sistemática contra estouro de buffer (*buffer overflow*).

2. Modelagem dos Dados, Structs e Campos

A modelagem dos dados atende rigorosamente aos limites estabelecidos no regulamento do Tema C, definindo restrições explícitas de tipos primitivos, faixas numéricas e tamanhos de vetores de caracteres com terminador nulo (`\0`).

2.1. O Registro do Livro (`Livro`)

```
#define TAM_TEXTO 80
#define ANO_MINIMO 1
#define ANO_MAXIMO 2026

typedef struct {
    int id;                         /* Identificador único e positivo (> 0) */
    char titulo[TAM_TEXTO];         /* Título da obra (até 79 caracteres úteis + '\0') */
    char autor[TAM_TEXTO];          /* Nome do autor (até 79 caracteres úteis + '\0') */
    int ano_publicacao;             /* Ano de publicação (inteiro entre 1 e 2026) */
} Livro;
```

- `id`: Inteiro estritamente positivo (`id > 0`). Valores ≤ 0 são rejeitados de imediato com o código `BST_ERRO_ID_INVALIDO`.
- `titulo` e `autor`: Vetores de caracteres de tamanho 80 (até 79 caracteres úteis). Rejeitam-se cadeias vazias (`""`), strings contendo apenas espaços em branco (`isspace()`) ou excesso de buffer.
- `ano_publicacao`: Inteiro restrito aos limites 1 ≤ ano ≤ 2026. Valores externos são rejeitados.

2.2. Nó da BST e Estrutura do Grafo

```
typedef struct NoBST {
    Livro livro;
    struct NoBST *esq;
    struct NoBST *dir;
} NoBST;

#define MAX_VERTICES 100

typedef struct {
    int ids_vertices[MAX_VERTICES];         /* Mapeia slot [0..99] -> ID arbitrário */
    bool ativo[MAX_VERTICES];               /* Booleano indicando ocupação ativa do slot */
    int adj[MAX_VERTICES][MAX_VERTICES];    /* Matriz de adjacência (0 ou 1) */
    int num_vertices;                       /* Quantidade de vértices ativos */
    int num_arestas;                        /* Quantidade total de arestas */
} Grafo;
```

3. Justificativa Técnica da Escolha das Estruturas de Dados

- **BST para Catálogo Primário:** Permite busca, inserção e remoção em tempo médio O(log n) com custo de memória estritamente linear O(n). Além disso, fornece listagem ordenada por ID via percurso em-ordem sem custo de ordenação adicional.
- **Grafo com Matriz de Adjacência:** Com capacidade estipulada em 100 nós simultâneos, a matriz de adjacência consome apenas ~40 KB de memória e permite consulta direta de adjacência (u, v) em tempo estrito O(1), sem a sobrecarga de percurso de ponteiros em listas de adjacência.
- **Pilha Dinâmica Encadeada para Histórico:** Comporta-se de forma estritamente LIFO, refletindo a ordem inversa das operações concluídas com sucesso. O modelo dinâmico aloca memória sob demanda e garante inserção (push) e remoção (pop) em O(1).
- **Fila Dinâmica FIFO para BFS:** Garante exploração radial em camadas concêntricas (níveis radiais de distância) a partir de uma origem, com operações de enfileirar e desenfileirar em O(1).
- **Lista Encadeada com Ponteiro de Fim:** Armazena a sequência exata de nós descobertos na BFS. Ao manter um ponteiro `fim` no descritor da lista, inserções ocorrem em O(1).
- **Merge Sort Próprio sobre Vetor Auxiliar:** Garante estabilidade na ordenação e desempenho estritamente delimitado em O(n log n) em todos os casos (melhor, médio e pior), eliminando a vulnerabilidade quadrática $O(n^2)$ do Quick Sort em cenários desfavoráveis.

4. Árvore Binária de Busca (BST) e Análise de Casos de Remoção

A BST indexa o catálogo pelo campo `id`. O algoritmo de remoção completo implementado em `bst_remover` resolve os três casos clássicos da estrutura:

1. **Caso 1: Nó Folha (0 Filhos):** O nó não possui descendentes (`esq == NULL && dir == NULL`). A memória é liberada com `free()` e o ponteiro no nó pai é atualizado para `NULL`.
2. **Caso 2: Nó com Apenas Um Filho:** O nó possui apenas a subárvore esquerda ou apenas a direita. O ponteiro do pai é redirecionado diretamente para esse filho único e o nó intermediário é liberado.
3. **Caso 3: Nó com Dois Filhos:** O nó possui ambas as subárvores. Localiza-se o **sucessor em-ordem** (menor elemento da subárvore direita via `bst_minimo(atual->dir)`). Os dados do sucessor são copiados para uma variável local, o sucessor é removido recursivamente da subárvore direita (onde cairá necessariamente no Caso 1 ou 2), e os dados do nó atual são substituídos pelos do sucessor. Isso suporta inclusive a remoção da raiz da árvore.

4.1. Análise da Altura e Degeneração

O desempenho de busca, inserção e remoção é linearmente dependente da altura h da árvore: T(n) = O(h). Em inserções aleatórias, a altura esperada é $h \approx \lfloor \log_2 n \rfloor$, proporcionando desempenho logarítmico O(log n). Contudo, em inserções estritamente ordenadas (por exemplo, IDs 10, 20, 30, 40...), a BST degenera em uma lista simplesmente encadeada de altura h = n - 1, degradando o tempo de operação para O(n).

5. Grafo, Mapeamento Compacto, Rollback e Busca em Largura (BFS)

5.1. Mapeamento de ID Arbitrário para Índice Compacto

Para desacoplar a matriz de adjacência de tamanho 100 dos valores de ID arbitrários do catálogo (ex: ID 5000), o módulo `grafo.c` gerencia um vetor `ids_vertices[100]` e flags `ativo[100]`. Ao remover um vértice, todas as suas arestas incidentes são zeradas, a flag `ativo` é desligada e o slot torna-se disponível para reutilização em novos cadastros.

5.2. Transações Integradas e Rollback

A função `sistema_cadastrar_livro` executa um protocolo transacional atômico: insere o livro na BST e tenta criar o vértice correspondente no Grafo. Caso o Grafo tenha atingido a sua capacidade máxima de 100 vértices, a inserção na BST é imediatamente desfeita (*rollback* via `bst_remover`), impedindo qualquer inconsistência ou registro órfão.

5.3. Busca em Largura (BFS)

A BFS implementada em `grafo_bfs` cumpre rigorosamente as especificações do regulamento:

- **Marcação no Enqueue:** O nó é marcado como visitado no momento exato em que entra na Fila, prevenindo duplicações.
- **Vizinhos em Ordem Crescente de ID:** Em cada vértice, todos os vizinhos conectados não visitados são ordenados por ID antes de serem enfileirados.
- **Origem Isolada:** Produz na lista encadeada apenas o seu próprio nó com distância zero.
- **Complexidade na Matriz:** O custo temporal da BFS sobre matriz de adjacência é $O(V^2)$, pois cada um dos V vértices varre uma linha de tamanho V da matriz.

6. Estruturas Auxiliares, Histórico e Algoritmo Próprio de Ordenação

6.1. Regra do Histórico na Pilha

A Pilha dinâmica encadeada armazena exclusivamente alterações de estado concluídas com sucesso (cadastro de livro, remoção de livro, adição de conexão e remoção de conexão). Operações de leitura e consulta não modificam a pilha. A exibição de histórico (opção 11) é não destrutiva, inspecionando até as 5 ações mais recentes sem desempilhar elementos.

6.2. Merge Sort sobre Vetor Auxiliar de Cópias

Para listar os livros ordenados por título (opção 5), os nós da BST são exportados por valor para um vetor alocado dinamicamente. O Merge Sort próprio ordena os registros de acordo com o critério:

1. Ordem alfabética de título sensível a maiúsculas e minúsculas via `strcmp(a->titulo, b->titulo)`;
2. Em caso de empate textual estrito (`strcmp == 0`), desempate por `id` crescente (`a->id < b->id`).

Como o vetor consiste em cópias independentes e é desalocado ao final da exibição, a BST original permanece completamente intacta e indexada por ID.

7. Tabelas de Análise de Complexidade Temporal e Espacial (Big-O)

Módulo / Estrutura	Operação	Melhor Caso (Tempo)	Caso Médio (Tempo)	Pior Caso (Tempo)	Complexidade Espacial
BST	Inserção (`bst_inserir`)	O(1) (raiz vazia)	O(log n)	O(n) (degenerada)	O(1) por nó
BST	Busca por ID (`bst_buscar`)	O(1) (na raiz)	O(log n)	O(n) (degenerada)	O(1)
BST	Remoção (`bst_remover`)	O(1) (folha raiz)	O(log n)	O(n) (degenerada)	O(1)
BST	Percurso Em-Ordem	O(n)	O(n)	O(n)	O(h) pilha de recursão
BST	Menor/Maior ID	O(1)	O(log n)	O(n)	O(1)
Grafo	Inserção de Vértice	O(1)	O(V)	O(V)	$O(V^2)$ matriz estática
Grafo	Remoção de Vértice	O(V)	O(V)	O(V)	O(1)
Grafo	Adição / Remoção de Aresta	O(1)	O(1)	O(1)	O(1)
Grafo	Busca em Largura (BFS)	$O(V^2)$	$O(V^2)$	$O(V^2)$	O(V) fila + lista
Grafo	Existência de Caminho	O(1) (mesmo nó)	$O(V^2)$	$O(V^2)$	O(V) visitados
Pilha	Push / Pop / Peek	O(1)	O(1)	O(1)	O(1) por nó
Fila	Enqueue / Dequeue	O(1)	O(1)	O(1)	O(1) por nó
Lista	Inserção no Fim	O(1)	O(1)	O(1)	O(1) por nó
Ordenação	Merge Sort Próprio	O(n log n)	O(n log n)	O(n log n)	O(n) auxiliar
Ordenação	Quick Sort Próprio	O(n log n)	O(n log n)	$O(n^2)$ (pior pivô)	O(log n) recursão

8. Relatório Completo dos Testes Reproduzíveis

Cenário	Dados de Entrada	Operação Testada	Resultado Esperado	Resultado Obtido	Status
C01	ID = 10, Titulo = "Dom Casmurro", 1899	Inserção na BST e Grafo	Registro e vértice alocados com sucesso	Sucesso em ambas as estruturas	APROVADO
C02	ID = 10 (já existente)	Inserção duplicada	Rejeição com BST_ERRO_DUPLICADO	Rejeição sem alteração na árvore	APROVADO
C03	ID = 0 e ID = -5	Inserção com ID inválido	Rejeição com BST_ERRO_ID_INVALIDO	Rejeição sem alteração na árvore	APROVADO
C04	ID = 999 (inexistente)	Remoção na BST	Retorno BST_ERRO_NAO_ENCONTRADO	Árvore permanece inalterada	APROVADO
C05	Nó folha (ID 20)	Remoção Caso 1 (0 filhos)	Ponteiro pai vira NULL; nó liberado	Nó removido; contagem decrementada	APROVADO
C06	Nó c/ 1 filho (ID 30)	Remoção Caso 2 (1 filho)	Subárvore filha promovida diretamente	Nó 30 removido; filho promovido	APROVADO
C07	Nó c/ 2 filhos (ID 50 - Raiz)	Remoção Caso 3 (2 filhos)	Sucessor 60 assume a nova raiz	Raiz atualizada para 60 com sucesso	APROVADO
C08	Vetor de 5 livros	Merge Sort por título c/ desempate ID	Vetor ordenado; BST original intacta	Vetor ordenado; BST inalterada	APROVADO
C09	Aresta (5000, 1200)	Adição de aresta no Grafo	Aresta simétrica criada em adj[u][v]	Conexão ativa em ambos os sentidos	APROVADO
C10	Aresta (1200, 5000)	Adição duplicada inversa	Retorno GRAFO_ERRO_ARESTA_DUPLICADA	Rejeição sem duplicar aresta	APROVADO
C11	Aresta (5000, 5000)	Tentativa de laço reflexivo	Retorno GRAFO_ERRO_LACO	Rejeição sem modificar matriz	APROVADO
C12	Vértice 100 c/ 3 arestas	Remoção de vértice	3 arestas zeradas e slot liberado	Arestas limpas; slot reciclado	APROVADO
C13	Vizinhos 50, 20, 40, 30	BFS a partir do nó 10	Ordem de visita: 10, 20, 30, 40, 50	Ordem estritamente crescente	APROVADO
C14	Vértice 9999 (isolado)	BFS com origem isolada	Lista contendo apenas [9999] dist 0	Lista gerada com nó único	APROVADO
C15	Componentes desconectados	Verificação de	Retorno existe = false	Ausência de caminho confirmada	APROVADO

Cenário	Dados de Entrada	Operação Testada	Resultado Esperado	Resultado Obtido	Status
		caminho			
C16	ID ausente (999)	Verificação de caminho	Retorno GRAFO_ERRO_VERTICE_NAO_ENCONTRADO	Erro reportado sem falso silencioso	APROVADO
C17	100 livros cadastrados	Tentativa de inserir 101º livro	Grafo cheio e Rollback na BST	BST e Grafo intactos com 100 nós	APROVADO
C18	Textos "" e " "	Validação de entrada textual	Rejeição de vazios e apenas espaços	Rejeitado com código de erro de I/O	APROVADO
C19	Texto com > 79 caracteres	Leitura via fgets com buffer 80	Descarte do excesso no stream	Buffer limpo sem contaminar stdin	APROVADO
C20	Ano = 0, -10 e 2027	Validação do ano de publicação	Rejeição fora da faixa [1, 2026]	Rejeitado com IO_ERRO_ANO_INVALIDO	APROVADO
C21	9 alterações de estado	Consulta da Pilha de Histórico	Exibição de 5 mais recentes s/ pop	5 ações exibidas; tamanho manteve 9	APROVADO
C22	Buscas e listagens	Verificação de estado da Pilha	Tamanho da pilha inalterado	Pilha permaneceu intacta	APROVADO

9. Conclusão e Limitações Conhecidas

O projeto atendeu a 100% dos requisitos estipulados no edital, comprovando a eficácia da combinação entre Árvores Binárias de Busca, Grafos, Filas e Pilhas no desenvolvimento de sistemas robustos em baixo nível. O sistema alcançou zero vazamentos de memória (*memory leaks*) e zero avisos de compilação sob `-Wall -Wextra -pedantic -std=c99`. Como limitações conhecidas, apontam-se a ausência de balanceamento automático da BST (suscetível a degeneração O(n) em casos patológicos de inserção ordenada) e a capacidade máxima nominal de 100 vértices imposta pela matriz estática do grafo.

10. Referências Bibliográficas

1. CORMEN, Thomas H. et al. *Algoritmos: Teoria e Prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.
2. SEDGEWICK, Robert; WAYNE, Kevin. *Algorithms*. 4th ed. Boston: Addison-Wesley, 2011.
3. ZIVIANI, Nivio. *Projeto de Algoritmos com Implementações em Pascal e C*. 3. ed. Cengage Learning, 2011.
4. KERNIGHAN, Brian W.; RITCHIE, Dennis M. *C: A Linguagem de Programação Padrão ANSI*. Rio de Janeiro: Campus, 1990.
5. CASTRO, Heber. *Especificação do Projeto Prático Individual de Estrutura de Dados — Tema C (Biblioteca)*. 2026.

11. Declaração Transparente do Uso de Ferramentas de Inteligência Artificial

O autor declara que utilizou ferramentas de Inteligência Artificial Generativa (Google DeepMind / Antigravity com modelos Gemini) durante o desenvolvimento deste trabalho acadêmico. O uso compreendeu auxílio na estruturação inicial de diretórios, geração do Makefile multiplataforma, sugestão de cenários de teste de borda e revisão ortográfica/formatação do relatório. Toda a lógica de algoritmos, ponteiros, gestão de memória e testes foi validada, compreendida e inspecionada pelo discente.